

Fast algorithm for the constrained longest common subsequence problem

SEBASTIAN DEOROWICZ ^a

^aInstitute of Computer Science,
Silesian University of Technology,
Akademicka 16, 44-100 Gliwice, Poland,
e-mail: Sebastian.Deorowicz@polsl.pl

Received 17 May 2007. Revised 4 October 2007. Accepted 21 October 2007.

Abstract: The problem of finding the constrained longest common subsequence (CLCS) for the sequences A and B with respect to the sequence P was introduced recently. Its goal is to find the longest subsequence of A and B such that P is a subsequence of it. The best known algorithms for its solving require time of order of a product of the sequence lengths. We introduce a novel approach in which time and space complexities depend also on the number of matches between A , B , and P . The time complexity is better for typical parameter settings and never worse.

Keywords: longest common subsequence, constrained longest common subsequence, string matching

1. Introduction

In various applications, there is a need to assess the similarity of two sequences, which can be defined in several ways. One of the most commonly used similarity measures is the length of their longest common subsequence (LCS)¹. In this problem, we ask for the longest sequence which is a subsequence of both sequences (a subsequence is obtained from a sequence by deleting zero or more symbols). The problem is well studied [2, 3, 6, 10] and is used in many applications, like DNA and protein analysis, text information retrieval, file comparing, music information retrieval, or spelling correction.

There are also several generalizations of the LCS and one of the recent is the *constrained longest common subsequence* problem (CLCS) [12]. In this problem, we deal

¹Some other possibilities, like *edit distance*, *best alignment* are discussed, e.g., in [2, 6].

with three sequences, A , B , and P , and ask for the longest such a sequence that is a subsequence of both A and B and contains P as a subsequence.

The origins of CLCS lay in bioinformatics, where sequences containing DNA and proteins are compared. In the classical LCS, we look for the longest subsequence and sometimes the obtained result is of little biological value. This is caused by the fact that we have no possibility to use any prior, biological, knowledge of the sequences. Tang *et al.* [13] give the following example. In the alignment of RNase sequences, it is known that sequences contain three active-site residues, His(H), Lyn(K), His(H), that are essential for RNA degrading. Therefore, biologists often are interested only in the subsequence in which these three residues occur in the given order (the sequence HKH is constrained).

Currently, the best algorithms for the CLCS problem need $O(mnr)$ time, where n , m , r are lengths of respectively A , B , and P . In this paper, we propose a Hunt–Szymanski-motivated improvement for those algorithms obtaining the algorithm of complexity $O(r(ml + d) + n)$, where l is the length of the LCS for A and B , and d is the number of matches between A and B . Easy to notice, our algorithm is never worse than $O(mnr)$ in time. We also show that its space complexity is attractive.

In Section 2., we introduce the necessary terms and define the CLCS problem. We also make a brief review of the existing methods. In Section 3., we introduce our method and then, in Section 5. we evaluate it in practice. Some technical details and algorithm complexity are discussed in Section 4. The last section concludes the paper.

2. Definitions and background

Let us have three sequences: $A = a_1, a_2, \dots, a_n$, $B = b_1, b_2, \dots, b_m$, and $P = p_1, p_2, \dots, p_r$, where $r \leq \min(m, n)$. Each of them is composed of symbols over an alphabet Σ of size σ . A sequence X' is a subsequence of X when it can be obtained from X by removing zero or more symbols. The LCS problem for A and B is to find the longest possible sequence C being a subsequence of both A and B . The CLCS problem is a generalization of the LCS because we have an additional requirement that P has to be a subsequence of C .

Since the problem is symmetric, we can assume without a loss of generality that $m \leq n$. We say that we have a *match* for the pair (i, j) iff $a_i = b_j$. We call a match *strong* iff for the triple (i, j, k) holds $a_i = b_j = p_k$. For simplicity we use the notation X_s for $x_1, x_2, \dots, x_s$.

Tsai in his paper [12] introduced the CLCS problem and presented an algorithm based on dynamic programming running in $O(m^2n^2r)$ time and space. Those complexities make the algorithm completely impractical. Further works showed that the CLCS problem is in fact much simpler. Peng [11], Arslan and Egecioğlu [1], and Chin *et al.* [5]

presented similar algorithms of time and space complexities $O(mnr)$ (in both worst and average-case). Chin *et al.* showed also that their method can be modified using the well-known Hirschberg technique [8] to obtain only $O(mr)$ storage while the time complexity is unchanged. Some other $O(mnr)$ -time algorithms were presented by Chin *et al.* [4] and He and Arslan [7]. The space complexity of Chin *et al.* approach is $O(mn)$, while of He and Arslan approach is $O(m + \alpha)$, where α is some parameter, usually much smaller than mn . (In fact, the last two algorithms solve the *constrained pairwise sequence alignment* problem, but the CLCS problem can be trivially transformed to it.) All the mentioned algorithms use dynamic programming.

k=0													k=1													k=2													k=3												
i 0 1 2 3 4 5 6 7 8 9 10 11													i 0 1 2 3 4 5 6 7 8 9 10 11													i 0 1 2 3 4 5 6 7 8 9 10 11													i 0 1 2 3 4 5 6 7 8 9 10 11												
j ABAADACBAABC													j ABAADACBAABC													j ABAADACBAABC													j ABAADACBAABC												
0C	0	0	0	0	0	1	1	1	1	1	1	1	0C	-	-	-	-	-	1	1	1	1	1	1	1	0C	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		
1B	0	1	1	1	1	1	1	2	2	2	2	2	1B	-	-	-	-	-	1	2	2	2	2	2	2	1B	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		
2C	0	1	1	1	1	1	2	2	2	2	2	3	2C	-	-	-	-	-	2	2	2	2	2	3	3	2C	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		
3B	0	1	1	1	1	1	2	3	3	3	3	3	3B	-	-	-	-	-	2	3	3	3	3	3	3	3B	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-			
4D	0	1	1	1	2	2	2	3	3	3	3	3	4D	-	-	-	-	-	2	3	3	3	3	3	3	4D	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-			
5A	1	1	2	2	2	3	3	3	4	4	4	4	5A	-	-	-	-	-	2	3	4	4	4	4	4	5A	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-			
6A	1	1	2	3	3	3	3	3	4	5	5	5	6A	-	-	-	-	-	2	3	4	5	5	5	5	6A	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		
7D	1	1	2	3	4	4	4	4	4	5	5	5	7D	-	-	-	-	-	2	3	4	5	5	5	5	7D	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		
8C	1	1	2	3	4	4	5	5	5	5	5	6	8C	-	-	-	-	-	5	5	5	5	5	6	6	8C	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
9D	1	1	2	3	4	4	5	5	5	5	5	6	9D	-	-	-	-	-	5	5	5	5	5	6	6	9D	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		
10B	1	2	2	3	4	4	5	6	6	6	6	6	10B	-	-	-	-	-	5	6	6	6	6	6	6	10B	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		
11A	1	2	3	3	4	5	5	6	7	7	7	7	11A	-	-	-	-	-	5	6	7	7	7	7	7	11A	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-		

Fig. 1. Example of the algorithm by Chin *et al.* for $A = \text{ABAADACBAABC}$, $B = \text{CBCBDAADCDBA}$, $P = \text{CBB}$. (Grayed cells denote strong matches and ‘-’ symbols denote $-\infty$ values.)

3. Proposed algorithm

We start the presentation of our algorithm from the recursive equation by Chin *et al.* [5] (an example of the algorithm is shown in Figure 1), which they introduced to calculate the CLCS:

$$M(i, j, k) = \begin{cases} M(i-1, j-1, k-1) + 1 & \text{if } i, j, k > 0 \\ & \text{and } a_i = b_j = p_k, \\ M(i-1, j-1, k) + 1 & \text{if } i, j > 0, a_i = b_j \\ & \text{and } (k = 0 \text{ or } a_i \neq p_k), \\ \max(M(i-1, j, k), M(i, j-1, k)) & \text{if } i, j > 0 \text{ and } a_i \neq b_j. \end{cases} \quad (1)$$

The initialization for $i = 0, 1, \dots, n$, $j = 0, 1, \dots, m$, and $k = 1, 2, \dots, r$ is done by:

$$M(i, 0, 0) = M(0, j, 0) = 0 \text{ and } M(0, j, k) = M(i, 0, k) = -\infty. \quad (2)$$

Each matrix cell $M(i, j, k)$ stores the length of the CLCS for A_i, B_j, P_k and the values of M are nondecreasing while the indexes grow, which allows us to rewrite Equation 1 in the equivalent form:

$$M(i, j, k) = \begin{cases} \max_{\substack{0 \leq i' < i \\ 0 \leq j' < j}} M(i', j', k-1) + 1 & \text{if } i, j, k > 0 \\ & \text{and } a_i = b_j = p_k, \\ \max_{\substack{0 \leq i' < i \\ 0 \leq j' < j}} M(i', j', k) + 1 & \text{if } i, j > 0, a_i = b_j \\ & \text{and } (k = 0 \text{ or } a_i \neq p_k), \\ \max_{\substack{0 \leq i' \leq i \\ 0 \leq j' \leq j \\ (i' \neq i \vee j' \neq j)}} M(i', j', k) & \text{if } i, j > 0 \text{ and } a_i \neq b_j. \end{cases} \quad (3)$$

To calculate the CLCS length we can restrict to triples (i, j, k) for which we have a match (not necessarily a strong one) since for non-match triples we make a copy of some cell, without increasing the value stored in it. Therefore for matches we can write:

$$M(i, j, k) = \begin{cases} \max_{\substack{0 \leq i' < i \\ 0 \leq j' < j \\ a_{i'} = b_{j'}}} M(i', j', k-1) + 1 & \text{if } i, j, k > 0 \\ & \text{and } a_i = b_j = p_k, \\ \max_{\substack{0 \leq i' < i \\ 0 \leq j' < j \\ a_{i'} = b_{j'}}} M(i', j', k) + 1 & \text{if } i, j > 0, a_i = b_j \\ & \text{and } (k = 0 \text{ or } a_i \neq p_k). \end{cases} \quad (4)$$

For the above equation, we have to specify the boundary conditions, since there are no matches for $i = 0$ or $j = 0$. We do this by adding a pseudo non-strong match $(0, 0)$ and the values of M are $M(0, 0, 0) = 0$ and $M(0, 0, k) = -\infty$ for $1 \leq k \leq r$.

In our approach, the values of M are computed level-wise, so we will start describing our method for a fixed value k , i.e., for single level. Together with M we compute also:

$$T(i, j, k) = \max_{\substack{0 \leq i' < i \\ 0 \leq j' < j \\ a_{i'} = b_{j'}}} M(i', j', k) + 1,$$

for all matches. These values are used to fill the $M(i, j, k)$ for non-strong matches. For strong matches they are superfluous in the current level, however, may be necessary while computing the next level.

To compute the values of T we employ a variant of the Hunt–Szymanski scheme for the LCS problem [9]. We process the matrix row-wise and in each row from left to right. For each distinct value x stored in M (so-called, M -value) we store in the list L the pair: x and the index of the leftmost column storing x , for the rows processed so far. Each time we meet a match (i, j) we look for the largest M -value with column index lower than i and use it to compute $T(i, j, k)$. Then, we use $T(i, j, k)$ as the value of $M(i, j, k)$ for

non-strong matches and $T(i, j, k - 1)$ for strong matches. After processing the row we update the list L in a single pass over it. (Figure 2 illustrates the contents of L during the row-wise processing of the M matrix for level 0.)

		i	1	2	3	4	5	6	7	8	9	10	11	12	
		j	A	B	A	A	D	A	C	B	A	A	B	C	
1	C								1					1	$L = \langle 7, 1 \rangle$
2	B		1							2			2		$L = \langle 2, 1 \rangle, \langle 8, 2 \rangle$
3	C								2					3	$L = \langle 2, 1 \rangle, \langle 7, 2 \rangle, \langle 12, 3 \rangle$
4	B		1							3			3		$L = \langle 2, 1 \rangle, \langle 7, 2 \rangle, \langle 8, 3 \rangle$
5	D					2									$L = \langle 2, 1 \rangle, \langle 5, 2 \rangle, \langle 8, 3 \rangle$
6	A	1		2	2		3				4	4			$L = \langle 1, 1 \rangle, \langle 3, 2 \rangle, \langle 6, 3 \rangle, \langle 9, 4 \rangle$
7	A	1		2	3		3				4	5			$L = \langle 1, 1 \rangle, \langle 3, 2 \rangle, \langle 4, 3 \rangle, \langle 9, 4 \rangle, \langle 10, 5 \rangle$
8	D					4									$L = \langle 1, 1 \rangle, \langle 3, 2 \rangle, \langle 4, 3 \rangle, \langle 5, 4 \rangle, \langle 10, 5 \rangle$
9	C							5						6	$L = \langle 1, 1 \rangle, \langle 3, 2 \rangle, \langle 4, 3 \rangle, \langle 5, 4 \rangle, \langle 7, 5 \rangle, \langle 12, 6 \rangle$
10	D					4									$L = \langle 1, 1 \rangle, \langle 3, 2 \rangle, \langle 4, 3 \rangle, \langle 5, 4 \rangle, \langle 7, 5 \rangle, \langle 12, 6 \rangle$
11	B		2							6			6		$L = \langle 1, 1 \rangle, \langle 2, 2 \rangle, \langle 4, 3 \rangle, \langle 5, 4 \rangle, \langle 7, 5 \rangle, \langle 8, 6 \rangle$
12	A	1		3	3		5				7	7			$L = \langle 1, 1 \rangle, \langle 2, 2 \rangle, \langle 3, 3 \rangle, \langle 5, 4 \rangle, \langle 6, 5 \rangle, \langle 8, 6 \rangle, \langle 9, 7 \rangle$

Fig. 2. Example of the algorithm calculating M matrix for $A = \text{ABAADACBAABC}$, $B = \text{CBCBDAADCDBA}$, and $k = 0$. The current content of the list L after finishing each row is also presented.

After filling out the whole matrix M , we can find the largest value stored in it, which is the length of the CLCS. In order to find also the CLCS, we need a three-dimensional matrix, F , which stores in each cell (i, j, k) the pointer to the cell of M used to calculate the $M(i, j, k)$ value. To find the CLCS, we start from the cell of M with the maximal value and move backward according to the pointers. The values of a_i for the visited cells form the CLCS.

4. Algorithm's complexity and implementation details

There are three three-dimensional matrices and some other variables, which may look as an important disadvantage comparing to the classical algorithm by Chin *et al.* [5], which needs only one three-dimensional matrix. We, however, introduced most of them only for a clear presentation. The simplest optimization is to notice that we do not need the whole T matrix. We need only to store the current and the previous level of T . Moreover, the list L and the previous level of T store all the necessary data to calculate the values of the current level of M and T . The maximal value of M can be determined from the list L at the end of the algorithm. Therefore, we can resign from the M matrix. All the matches between A and B can be determined easily and we only need to precalculate a vector V_0 storing the number of appearances of all the alphabet symbols in A and the matrix V_1 with the positions of the symbols' appearance.

Both can be implemented in $O(\max(n, \sigma))$ space. The two-dimensional array T can be implemented in $O(d)$ memory, where d is the number of matches between A and B , since only d cells of it is used. The list L obviously occupies $O(l)$ space, where l is the LCS length of A and B . If we are interested only in the CLCS length, that is all we need to store and the overall memory consumption is $O(d + \max(n, \sigma))$. If we are also interested in the CLCS, we need also the F matrix, which can be implemented in $O(rd)$ memory, so the total space consumption is $O(rd + \max(n, \sigma))$.

The initialization of the vector V_0 and match-array V_1 takes $O(\max(n, \sigma))$ time, which is negligible if $\sigma = O(n)$. (The case of large σ will be discussed later.) The processing of each level consumes $O(ml + d)$ time, since for each row we browse the list from left to right and for each match we perform at least one access at the list (but no more than $O(l)$ for all the matches in each row, because we never go backward in the list). The list must be also updated, but this can be done in the simple left-to-right scan over it. Therefore the maintenance of the list L takes $O(r(ml + d))$ time overall, where l is the LCS length of A and B , and d is the total number of matches for A and B . Each access to the M , T , F matrices takes only $O(1)$ time. We should also remember the initialization cost of the vector V_1 storing the positions of all the alphabet symbols, which is $O(n)$. This leads to the total time complexity $O(r(ml + d) + n)$.

Up to now, we neglected the alphabet size. If $\sigma = O(n)$ the above calculations are true. If σ is larger, we should change the algorithm a bit. We sort the symbols forming A , remove repetitions and relabel the symbols in both A and B . This leads to the additional time complexity term $O(m + n \log n) = O(n \log n)$ (in the comparison based model), so the total time complexity is $O(r(ml + d) + n \log n)$.

The complete pseudo-code for the calculation of the CLCS length is shown in Figure 3. In lines 01–04, we compute the positions of all the alphabet symbols in the sequence A . There are exactly n such positions, so the array V^1 can be implemented in $O(n)$ memory. Lines 05–33 contain the main loop, in which we traverse the matrices in a level-wise manner. The initialization of the M matrix is here simplified, and we only set the M -value for the pseudo-match at position $(0, 0, k)$ to 0 when we process the 0-level and $-\infty$, when the level is higher. In the loop 12–33, we browse the current level row-wise. For each row, after copying the pseudo-match, we move through all the row matches. In lines 15–21, we make a copy of the part of the list L^0 to L^1 . Then, we add a new match to the list L^1 (lines 22–28). In lines 29–32, we make a copy of the remaining elements of L^0 . Then, we exchange the lists L^0 and L^1 to make the newly created list the current one for the next row, and empty the other list. Figure 4 shows the pseudocode of the CLCS sequence computing procedure.

```

    {A[1..n], B[1..m], P[1..r]}
01 for  $i \leftarrow 1$  to  $\sigma$  do
02    $V^0[i] \leftarrow 0$ ;
03 for  $i \leftarrow 1$  to  $n$  do
04    $V^1[A[i]][V^0[A[i]]] \leftarrow i$ ;  $V^0[A[i]] \leftarrow V^0[A[i]] + 1$ ;
05 for  $k \leftarrow 0$  to  $r$  do
06   if  $k = 0$  then
07      $L^0[0].len \leftarrow 0$ ;
08   else
09      $L^0[0].len \leftarrow -\infty$ ;
10    $L^0[0].i \leftarrow -\infty$ ;  $L^0[0].j \leftarrow -\infty$ ;
11    $N^0 \leftarrow 1$ ;  $N^1 \leftarrow 0$ ;
12   for  $j \leftarrow 1$  to  $m$  do
13      $L^1[N^1] \leftarrow L^0[0]$ ;  $N^1 \leftarrow N^1 + 1$ ;  $p \leftarrow 1$ ;
14     for  $s \leftarrow 0$  to  $V^0[B[j]] - 1$  do
15        $i \leftarrow V^1[B[j]][s]$ ;
16       while  $p < N^0$  do
17         if  $L^0[p].i \geq i$  then
18           break;
19         else if  $L^1[N^1 - 1].len < L^0[p].len$  and  $L^0[p].len > 0$  then
20            $L^1[N^1] \leftarrow L^0[p]$ ;  $N^1 \leftarrow N^1 + 1$ ;
21          $p \leftarrow p + 1$ ;
22       if  $k > 0$  and  $B[j] = P[k]$  then
23          $v \leftarrow T[j][i]$ ;  $T[j][i] \leftarrow L^0[p - 1].len + 1$ ;
24       else
25          $v \leftarrow L^0[p - 1].len + 1$ ;  $T[j][i] \leftarrow v$ ;
26        $F[k][j][i].i \leftarrow L^0[p - 1].i$ ;  $F[k][j][i].j \leftarrow L^0[p - 1].j$ ;
27       if  $L^1[N^1 - 1].len < v$  then
28          $L^1[N^1].i \leftarrow i$ ;  $L^1[N^1].j \leftarrow j$ ;  $L^1[N^1].len \leftarrow v$ ;  $N^1 \leftarrow N^1 + 1$ ;
29       while  $p < N^0$  and  $L^1[N^1 - 1].len \geq L^0[p].len$  do
30          $p \leftarrow p + 1$ ;
31       while  $p < N^0$  do
32          $L^1[N^1] \leftarrow L^0[p]$ ;  $N^1 \leftarrow N^1 + 1$ ;  $p \leftarrow p + 1$ ;
33        $L^0 \leftarrow L^1$ ;  $N^0 \leftarrow N^1$ ;  $N^1 \leftarrow 0$ ;
34 return  $L^0[N^0 - 1].len$ ;

```

Fig. 3. Algorithm computing the length of the CLCS

```

    {A[1..n], B[1..m], P[1..r]}
01  $i \leftarrow L^0[N^0 - 1].i$ ;  $j \leftarrow L^0[N^0 - 1].j$ ;  $k \leftarrow r$ ;
02 for  $len \leftarrow L^0[N^0 - 1].len$  downto 1 do
03    $R[len] \leftarrow B[j]$ ;
04   if  $k > 0$  and  $B[j] = P[k]$  then
05      $k \leftarrow k - 1$ ;
06      $i' \leftarrow F[k][j][i].i$ ;  $j' \leftarrow F[k][j][i].j$ ;
07      $i \leftarrow i'$ ;  $j \leftarrow j'$ ;
08 return  $R$ ;

```

Fig. 4. Algorithm computing the CLCS

5. Experimental results

The existing algorithms computing the CLCS in $O(mnr)$ time are closely related so we selected the one by Chin *et al.* [5] for practical experiments. The algorithm by Tsai [12] of complexity $O(m^2n^2r)$ is obviously the loser and we resign from including it in the experiments. Therefore, there are only two methods to compare: the one by Chin *et al.* and our. We implemented both algorithms in C++, compiled using Microsoft Visual C++ 2005 compiler², and run on a computer equipped with AMD Athlon 64 X2 5000+ processor (real 2600 MHz CPU clock) and 2GB RAM. Each time is a median of 51 executions.

In the first experiment, we fixed the length of the constrained sequence ($r = 16$) and evaluated the algorithms for various alphabet sizes (Fig. 5)³ (when the alphabet size grows the number of matches decreases). The results show that the speed of the Chin *et al.* method almost does not change when σ grows. It is what one would expect since this method always computes all the $O(mnr)$ cells of the dynamic programming matrix. The speed of our algorithm is highly dependent on the number of matches, which can be easily observed. Our method loses for small alphabets and wins for alphabets of size exceeding 5. The most popular application of the CLCS problem is in bioinformatics where the sequences store protein data. In such a case, $\sigma = 20$ and our method is about three times faster than the method by Chin *et al.*

In the second experiment (Fig. 6a), we fixed the alphabet size ($\sigma = 20$) and the constrained sequence length ($r = 16$), and compared the algorithms for various lengths of the two main sequences. We examined not only the algorithms computing the CLCS, but also their variants computing only the CLCS length. This is a simpler task, which can be done faster, since only a two-dimensional matrix is sufficient because no traceback to obtain the CLCS is made. Moreover, in our method, also the back pointers in matrix

²The /Ox switch was used for the speed optimization.

³The sequences are uniformly random.

cells are superfluous. The results show that the algorithms computing the CLCS lengths are about 30% faster than their CLCS-variants. It is interesting to notice, that our CLCS-computing method is over two times faster than the CLCS-length-computing method by Chin *et al.*

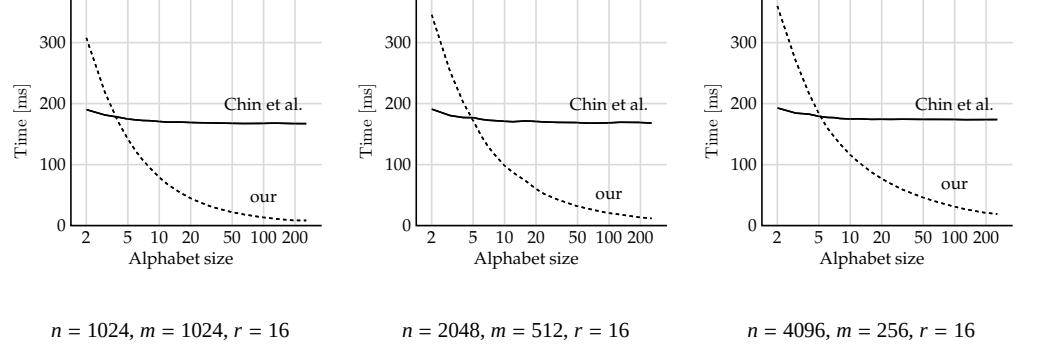


Fig. 5. Comparison of the examined algorithms for changing alphabet size.

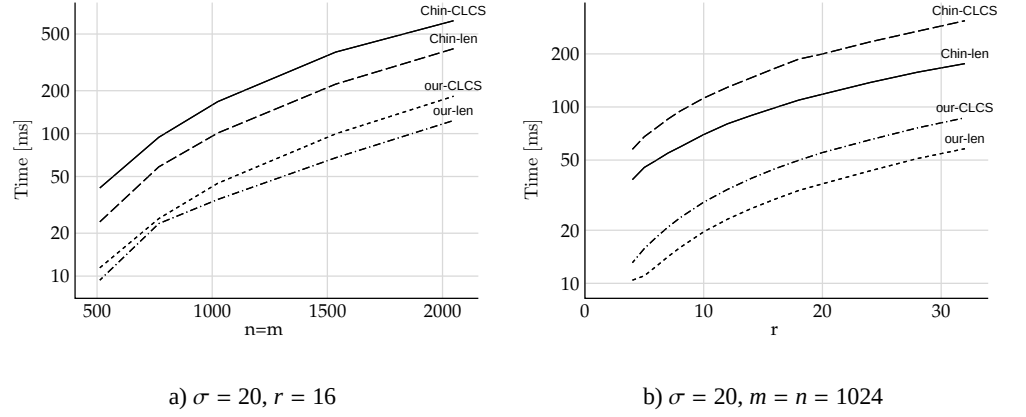


Fig. 6. Comparison of the examined algorithms for fixed alphabet size.

In the last experiment (Fig. 6b), we evaluated the influence of the constrained sequence length. There were no surprises and the results show that the times of the examined algorithms depend almost linearly on this parameter.

The memory requirements of the Chin *et al.* algorithm is steady and for the case $n = m = 1024, r = 16$ it consumes about 72.15 MB. The memory occupation of our method depend on the number of matches between A and B . For the same parameters ($n = m = 1024, r = 16$), it decreases from 70.03 MB for a binary alphabet, through 30.07 MB for the case $\sigma = 4$ and 7.13 MB for $\sigma = 20$ to 0.65 MB for $\sigma = 256$. (We did

not experiment with the Hirschberg-like memory optimization of the algorithm by Chin *et al.* since the time necessary to reduce the memory complexity is significant and the algorithm would be much slower than the classical one.)

6. Conclusions

We considered the constrained longest common subsequence problem (CLCS). The problem is rather new and only a few dynamic programming-based algorithms were presented so far. The best known ones are of $O(mnr)$ time complexity, where the factors are sequence lengths.

We presented the first algorithm exploiting sparsity of the dynamic programming matrix, making use of the well-known Hunt–Szymanski technique [9], developed for the LCS problem. The time complexity of the method is $O(r(ml + d) + n)$, where l is the LCS length for A and B and d is the number of matches between A and B . The memory requirements of our algorithm is $O(dr + \max(n, \sigma))$, where σ is the alphabet size, which is also attractive. The experimental results show that our algorithm is about three times faster than the examined $O(mnr)$ -time algorithm by Chin *et al.* for a typical case (i.e., $\sigma = 20$ which is the alphabet size of proteins).

Recently He and Arslan [7] showed an improvement to the base algorithm by Chin *et al.* [4]. They postulate to find the ‘entry’ and ‘exit’ points, i.e., the strong matches for which the values of M are computed relating on the previous level and the strong matches needed for the calculation of the next level of M . Thanks to this knowledge, one can restrict the calculation of each level of M to only this part which is used for computation of the next level and some useless computations are omitted. The gain in practice of this procedure depends on the contents of the sequences A , B , P , but the worst-case time complexity is still $O(mnr)$. Similar strategy can be easily adapted for our algorithm, since this is no problem to restrict to calculation of only part of the M , T , F matrices.

Acknowledgments

The author thanks Szymon Grabowski for his comments on the paper contents. Supported by Silesian University of Technology Research Project BK-233/Rau2/2007.

References

- [1] Arslan A.N., Egecioğlu Ö.: Algorithms for the constrained longest common subsequence problems. *International Journal of Foundations of Computer Science*. 16(6), pp. 1099-1109, 2005.

- [2] Apostolico A.: General pattern matchings. Chapter in Handbook of Algorithms and Theory of Computation, M.J. Atallah (Editor), Chapter 13, 1998.
- [3] Bergroth L., Hakonen H., Raita T.: A survey of longest common subsequence algorithms. In Proceedings of 7th International Symposium on String Processing Information Retrieval (SPIRE), Curuña, Spain, pp. 39-48, 2000.
- [4] Chin F.Y.L., Ho N.L., Lam T.W., Wong P.W.H., Chan M.Y.: Efficient Constrained Multiple Sequence Alignment with Performance Guarantee. Journal of Bioinformatics and Computational Biology, 3(1), pp. 1-18, 2005.
- [5] Chin F.Y.L., De Santis A., Ferrara A.L., Ho N.L., Kim S.K.: A simple algorithm for the constrained sequence problems. Information Processing Letters, 90, pp. 175-179, 2004.
- [6] Gusfield D.: Algorithms on strings, trees, and sequences. Cambridge University Press, USA (1999).
- [7] He D., Arslan A.N.: A Space-Efficient Algorithm for the Constrained Pairwise Sequence Alignment Problem. Genome Informatics, 16(2), pp. 237-246, 2005.
- [8] Hirschberg D.S.: Algorithms for the longest common subsequence problem. Journal of the ACM, 24, pp. 664-675, 1977.
- [9] Hunt J.W., Szymanski T.G.: A fast algorithm for computing longest common subsequences. Communications of the ACM 20(5), (1977), pp. 350-353.
- [10] Navarro G.: A Guided Tour to Approximate String Matching. ACM Computing Surveys, 33(1), pp. 31-88, 2001.
- [11] Peng Ch.-L.: An Approach for Solving the Constrained Longest Common Subsequence Problem. Master's Thesis, Department of Computer Science and Engineering, National Sun Yat-sen University, Taiwan, 2003.
<http://etd.lib.nsysu.edu.tw/ETD-db/ETD-search/getfile?URN=etd-0828103-125439&filename=etd-0828103-125439.pdf>
- [12] Tsai Y.-T.: The constrained common subsequence problem. Information Processing Letters, 88, pp. 173-176, 2003.
- [13] Tank C.Y., Lu C.L., Chang M.D.-T., Tsai Y.-T., Sun Y.-J., Chao K.-M., Chang J.-M., Chiou Y.-H., Wu C.-M., Chang H.-T., Chou W.-I.: Constrained multiple sequence alignment tool development and its application to RNase family alignment. In Proceedings of the first IEEE Computer Society Bioinformatics Conference (CSB 2002), pp. 127-137, 2002.

Szybki algorytm dla znalezienia najdłuższego kluczowego wspólnego podciągu

Streszczenie

Problem wyznaczania najdłuższego wspólnego podciągu ciągów A i B , którego podciągiem jest podciąg P (CLCS) pojawił się w literaturze stosunkowo niedawno. Złożoności obliczeniowe najlepszych znanych algorytmów są iloczynem długości wszystkich ciągów. W niniejszej

pracy przedstawione jest nowy algorytm rozwiązywania tego problemu, dla którego złożoność obliczeniowa zależy od liczby dopasowań pomiędzy ciągami A , B i P . Złożoność ta jest zwykle lepsza (a nigdy gorsza) niż najlepszych znanych do tej pory algorytmów.

σ